

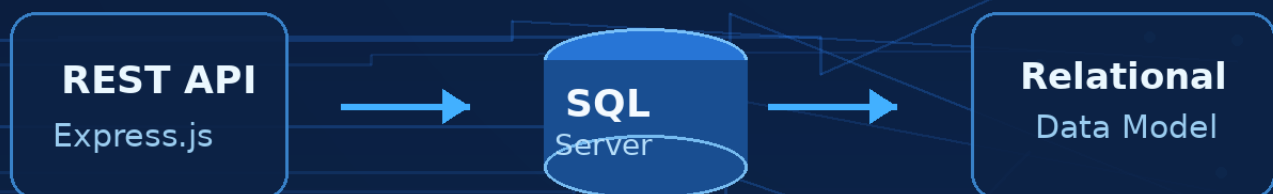


ClarityLoop DB

Persistent Decision Intelligence API

Database Integration Report

Node.js + Express + Microsoft SQL Server



PREPARED BY

Mustafa Tahir

Full Stack Development Intern
Decode Labs | Batch 2026

PROJECT FOCUS

Database Integration
Relational Schema Design
CRUD Operations
Validation & Transactions

INTERNSHIP PROJECT REPORT

Decode Labs - Full Stack Development Internship | Project 3

Executive Summary

ClarityLoop DB is a database-backed REST API developed for Decode Labs Project 3: Database Integration. The project replaces flat-file persistence with Microsoft SQL Server and connects a Node.js/Express backend to a normalized relational database. It demonstrates the core requirements of the internship milestone: database connectivity, schema design, persistent CRUD operations, and reliable data handling.

The solution models decisions, alternative options, post-decision reviews, reusable tags, and decision-tag associations across five related tables. The backend exposes REST endpoints for create, read, update, delete, and database health checks. Data integrity is reinforced through primary and foreign keys, check and unique constraints, cascading deletes, typed SQL parameters, request validation, and an explicit transaction for multi-table CREATE operations.

Manual verification was completed through SQL Server Management Studio, Node.js console output, and Postman. The evidence set covers schema creation, relationship queries, database connectivity, health checks, all four CRUD operations, deletion verification, and invalid-input handling.

Project Outcome

Project 3 requirements are met and exceeded: a relational SQL Server schema is connected to the backend, CRUD operations persist data, and proper data handling is demonstrated through constraints, validation, parameterized SQL and transaction control.

Project Information

Item	Details
Project	ClarityLoop DB - Persistent Decision Intelligence API
Intern	Mustafa Tahir
Role	Full Stack Development Intern
Organization	Decode Labs
Internship Track	Full Stack Development - Batch 2026
Milestone	Project 3 - Database Integration
Backend	Node.js + Express.js
Database	Microsoft SQL Server (ClarityLoopDB)
Testing	Postman + SSMS + Node.js connectivity checks
Report Date	August 2026

Table of Contents

Report navigation

Executive Summary..... 1

1. Introduction..... 3

2. Project Objectives and Scope..... 4

3. System Overview and Architecture..... 5

4. Database Design..... 6

5. Database and Backend Implementation..... 8

6. REST API Design..... 9

7. CRUD Operations..... 10

8. Testing and Validation..... 11

9. Evidence Screenshots..... 12

10. Key Engineering Decisions and Challenges..... 15

11. Learning Outcomes..... 16

12. Conclusion..... 17

Appendix A - Setup and Execution Guide..... 18

Appendix B - Submission Repository Structure..... 19

1. Introduction

1.1 Background

Project 3 in the Decode Labs Full Stack Development track focuses on persistence. The training brief requires the backend to be connected to a database so that application data is stored beyond the lifetime of a process. The minimum expectations are a simple schema, basic CRUD operations, and proper data handling.

ClarityLoop DB addresses this milestone by evolving an earlier decision-tracking backend from JSON-file storage to a relational SQL Server database. The application records a decision together with its context, reasoning, confidence level, candidate options, selected option, tags, and later review information.

1.2 Problem Statement

Flat-file persistence is easy to start with but becomes difficult to manage when records have multiple related entities and integrity rules. ClarityLoop needs durable storage that can represent one-to-many, one-to-one and many-to-many relationships while supporting reliable create, read, update and delete operations.

1.3 Proposed Solution

- Use Microsoft SQL Server as the persistent relational database.
- Model the decision domain using five related tables.
- Connect Node.js/Express to SQL Server through the mssql/msnodesqlv8 stack.
- Expose REST endpoints for CRUD operations and health verification.
- Use request validation, database constraints, parameterized SQL and transactions to protect data integrity.
- Verify implementation through SSMS queries, Postman requests and console-level connectivity tests.

2. Project Objectives and Scope

2.1 Objectives

- Design a relational database schema suitable for persistent decision records.
- Implement one-to-many, one-to-one and many-to-many relationships.
- Connect the Express backend to Microsoft SQL Server.
- Implement CREATE, READ, UPDATE and DELETE operations for decision records.
- Use typed parameters and validation instead of concatenating user input into SQL.
- Preserve consistency during multi-table inserts through transaction control.
- Test database connectivity, CRUD behavior, validation and relationship queries.
- Document the database design, API behavior, setup process and test evidence.

2.2 Scope

The current project is a local backend and database implementation. It does not include authentication, a public cloud deployment, a graphical frontend, automated integration tests, pagination, or a dedicated GET-by-ID endpoint. These are treated as future enhancements rather than Project 3 requirements.

2.3 Requirement Traceability

Decode Labs Requirement	Implementation	Evidence	Status
Connect backend with a database	Node.js/Express connected to ClarityLoopDB on SQL Server	Connection and health screenshots	Met
Design a simple database schema	Five relational tables with keys, constraints and relationships	Schema SQL + database tables screenshot	Met
Perform basic CRUD operations	POST, GET, PUT and DELETE decision endpoints	Postman CRUD screenshots	Met
Ensure proper data handling	Validation, parameterized SQL, transaction, cascading deletes	Code + 400 validation + delete verification	Met

3. System Overview and Architecture

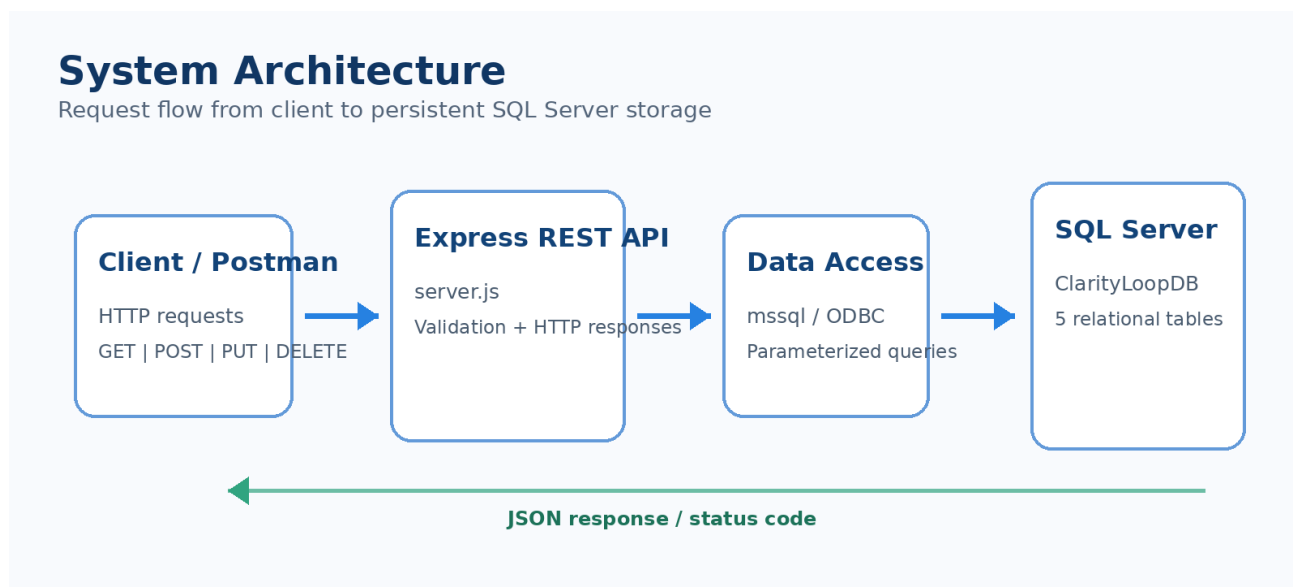


Figure 1. High-level ClarityLoop DB backend architecture.

3.1 Request Flow

1. A client such as Postman sends an HTTP request to the Express application.
2. The route handler validates required fields and input ranges.
3. The backend obtains a SQL Server connection pool from the database module.
4. Typed parameters are bound through the mssql library and the SQL statement is executed.
5. SQL Server applies relational constraints and persists the requested changes.
6. The API returns JSON with an appropriate HTTP status code or a controlled error response.

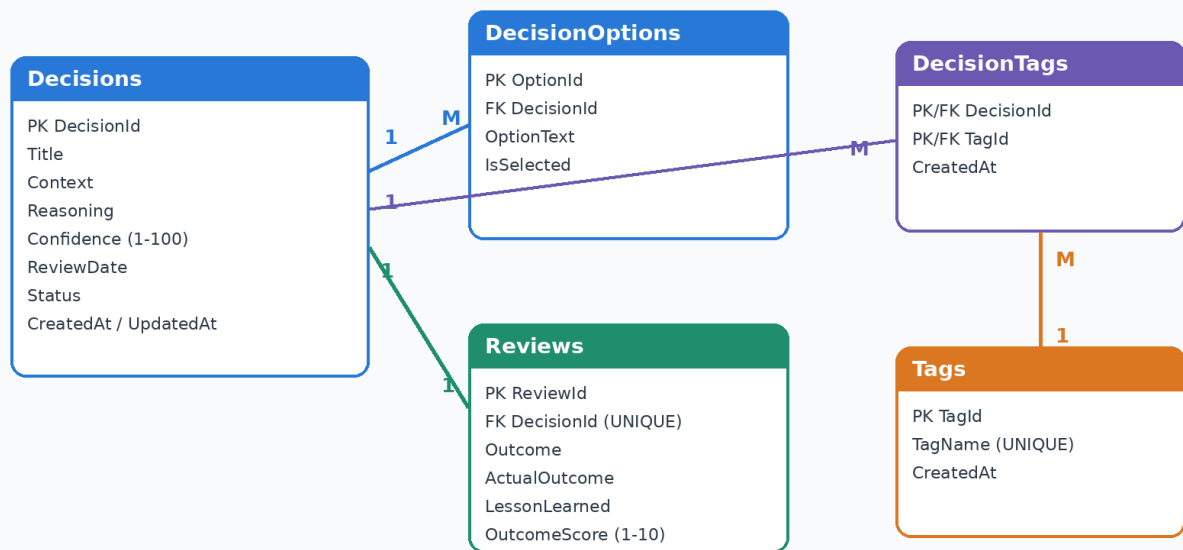
3.2 Technology Stack

Layer	Technology	Purpose
Runtime	Node.js	Runs the backend application.
Framework	Express.js	Defines REST routes and JSON request handling.
Database	Microsoft SQL Server	Stores persistent relational data.
Administration	SQL Server Management Studio	Schema execution, data inspection and query verification.
Database Driver	mssql + msnodesqlv8	Provides Node.js connectivity to SQL Server.
Connectivity	ODBC Driver 18 for SQL Server	Supports the configured local connection.
API Testing	Postman	Executes CRUD and validation test requests.
Version Control	Git / GitHub	Packages the final submission repository.

4. Database Design

ClarityLoopDB Relational Model

Five-table schema with 1:M, 1:1 and M:M relationships



Integrity features: primary keys, foreign keys, unique/check constraints, defaults, cascading deletes, filtered unique index.

Figure 2. ClarityLoopDB relational schema and cardinalities.

4.1 Table Responsibilities

Table	Role	Key Integrity Rules
Decisions	Main decision record containing context, reasoning, confidence, review date and status.	Confidence 1-100; status limited to pending-review/reviewed.
DecisionOptions	Stores candidate choices for each decision.	FK to Decisions; unique option text per decision; only one selected option.
Reviews	Stores the post-decision outcome and lesson learned.	Unique DecisionId creates a 1:1 relation; outcome and score constrained.
Tags	Stores reusable tag names.	Unique TagName; length check.
DecisionTags	Junction table connecting decisions and tags.	Composite primary key; cascading foreign keys.

4.2 Relationships and Integrity Controls

- One Decision to many DecisionOptions.
- One Decision to one Review because Reviews.DecisionId is UNIQUE.
- Many Decisions to many Tags through the DecisionTags junction table.
- Foreign keys enforce valid parent-child references.
- ON DELETE CASCADE removes dependent option, review and decision-tag rows when a decision is deleted.
- A filtered unique index ensures that no decision can have more than one selected option.
- Check constraints restrict confidence, decision status, review outcome and review score to valid ranges.
- Default values generate identifiers and UTC timestamps automatically.

4.3 Selected Schema Excerpt

```
CREATE TABLE dbo.Decisions (  
    DecisionId UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),  
    Title NVARCHAR(120) NOT NULL,  
    Context NVARCHAR(500) NOT NULL,  
    Reasoning NVARCHAR(1000) NOT NULL,  
    Confidence TINYINT NOT NULL,  
    ReviewDate DATE NULL,  
    Status NVARCHAR(20) NOT NULL DEFAULT 'pending-review',  
    CONSTRAINT CK_Decisions_Confidence  
        CHECK (Confidence BETWEEN 1 AND 100)  
);
```

Why this design matters

The database does more than store values. It also enforces business rules so invalid or inconsistent data is rejected even if an application bug attempts to insert it.

5. Database and Backend Implementation

5.1 Database Setup

7. Create or select the ClarityLoopDB database in Microsoft SQL Server.
8. Run the schema script to create Decisions, DecisionOptions, Reviews, Tags and DecisionTags.
9. Run the optional test-data script to insert a related decision, options, tags and review.
10. Execute the verification script to check row counts and relationship queries.
11. Confirm that the backend can open a connection to ClarityLoopDB.

5.2 Node.js SQL Server Connection

```
const sql = require("mssql/msnodesqlv8");

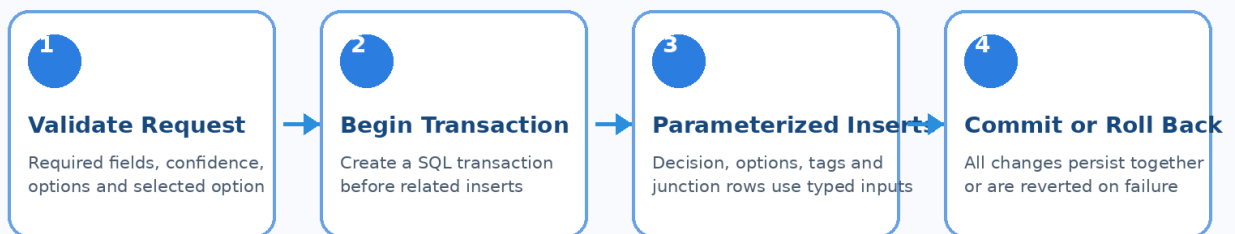
const config = {
  connectionString:
    "Driver={ODBC Driver 18 for SQL Server};" +
    "Server=localhost;" +
    "Database=ClarityLoopDB;" +
    "Trusted_Connection=Yes;" +
    "Encrypt=Optional;"
};
```

The local project uses Windows Authentication against a local SQL Server instance. The report intentionally does not include passwords, tokens or production credentials. A public deployment would require a remotely accessible SQL Server instance and deployment-safe secret management.

5.3 Data Handling Strategy

Reliable Data Handling

How CREATE protects data integrity across related inserts



Result: consistent relational state even when a multi-step operation fails.

Figure 3. Validation, parameterized queries and transaction-based CREATE flow.

6. REST API Design

6.1 Endpoint Summary

Method	Endpoint	Purpose	Success
GET	/api/health	Verify API and database connection.	200
GET	/api/v1/decisions	Retrieve all decision records.	200
POST	/api/v1/decisions	Create a decision with options and tags.	201
PUT	/api/v1/decisions/:id	Update core decision fields.	200
DELETE	/api/v1/decisions/:id	Delete a decision and cascade related rows.	200

6.2 Validation Rules

- Title, context and reasoning are required for create and update operations.
- Confidence must be an integer between 1 and 100.
- CREATE requires at least two decision options.
- The selected option must exist in the supplied options array.
- Blank tag strings are ignored after trimming and lowercasing.

6.3 Parameterized SQL

```
decisionRequest.input("Title", sql.NVarChar(120), title);  
decisionRequest.input("Context", sql.NVarChar(500), context);  
decisionRequest.input("Reasoning", sql.NVarChar(1000), reasoning);  
decisionRequest.input("Confidence", sql.TinyInt, confidence);
```

Typed parameters keep application values separate from SQL syntax and allow the driver to bind values according to the expected SQL Server data type.

7. CRUD Operations

7.1 Create

POST /api/v1/decisions creates the main decision first, then inserts its options and tags within the same SQL transaction. The selected option is marked through IsSelected, while tags are reused when they already exist.

7.2 Read

GET /api/v1/decisions returns all decision rows ordered by creation time in descending order. The response contains a success flag, record count and the retrieved data.

7.3 Update

PUT /api/v1/decisions/:id validates the updated payload, updates title, context, reasoning, confidence and review date, refreshes UpdatedAt, and returns the updated row. Unknown identifiers return 404 Not Found.

7.4 Delete

DELETE /api/v1/decisions/:id removes the decision and returns the deleted identifier and title. Related options, reviews and decision-tag rows are automatically removed through cascading foreign keys.

7.5 Transaction Safety

```
transaction = new sql.Transaction(pool);
await transaction.begin();

... insert decision ...
... insert options ...
... insert tags and decision-tag rows ...

await transaction.commit();

// on failure
await transaction.rollback();
```

8. Testing and Validation

Testing was performed manually using SQL Server Management Studio, Node.js console output and Postman. The test set verifies both successful operations and controlled failure behavior.

#	Test	Expected Result	Evidence	Result
1	Schema creation	All five tables created.	Database tables screenshot	PASS
2	Relationship query	Related decision/options/review/tags returned.	Test data relationships screenshot	PASS
3	Node.js DB connection	Backend connects to ClarityLoopDB.	Connection screenshot	PASS
4	Health endpoint	200 with database health payload.	API health screenshot	PASS
5	READ	200 and decision array.	CRUD Read screenshot	PASS
6	CREATE	201 and new DecisionId.	CRUD Create screenshot	PASS
7	Read after create	New record appears in collection.	Read-after-create screenshot	PASS
8	UPDATE	200 and modified decision values.	CRUD Update screenshot	PASS
9	DELETE	200 and deleted decision metadata.	CRUD Delete screenshot	PASS
10	Delete verification	Deleted record no longer appears.	Delete verification screenshot	PASS
11	Validation	400 for invalid request data.	Validation screenshot	PASS

9. Evidence Screenshots

The following screenshots are included as implementation evidence and correspond to the manual test matrix.

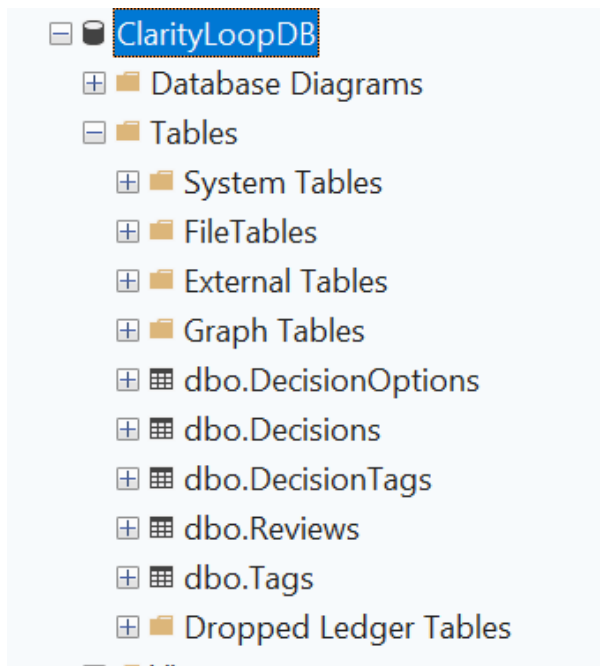


Figure 4. SQL Server tables created for ClarityLoopDB.

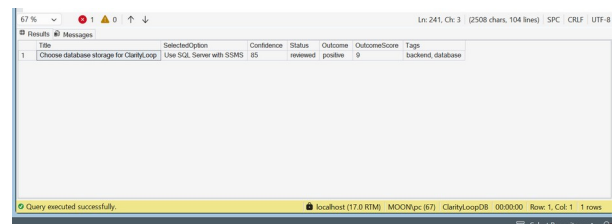


Figure 5. Relationship query using inserted test data.

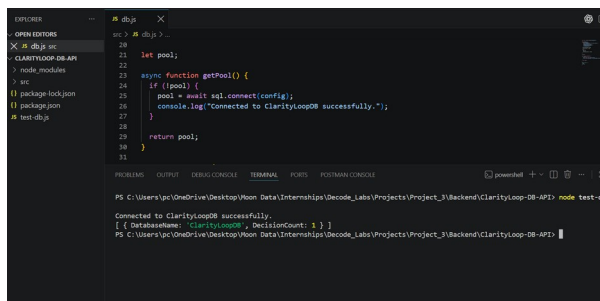


Figure 6. Successful Node.js to SQL Server connection.



Figure 7. API health endpoint confirming database availability.

9. Evidence Screenshots (continued)



Figure 8. GET /api/v1/decisions - READ operation.

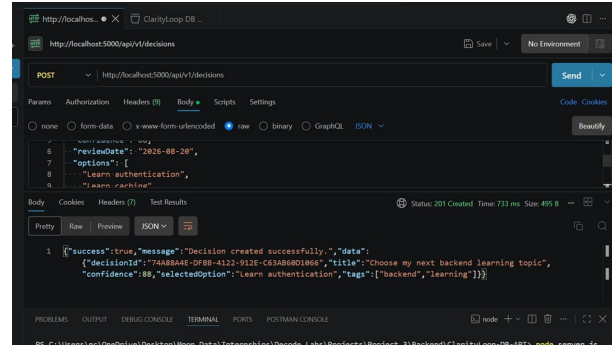


Figure 9. POST /api/v1/decisions - CREATE operation.

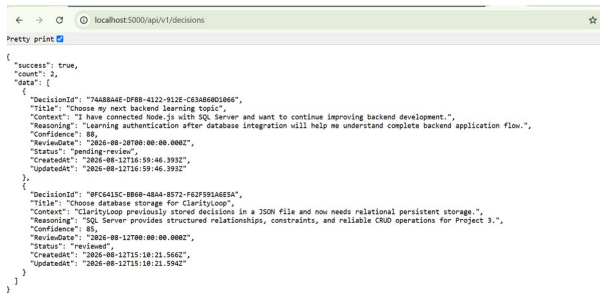


Figure 10. READ confirms the newly created decision.

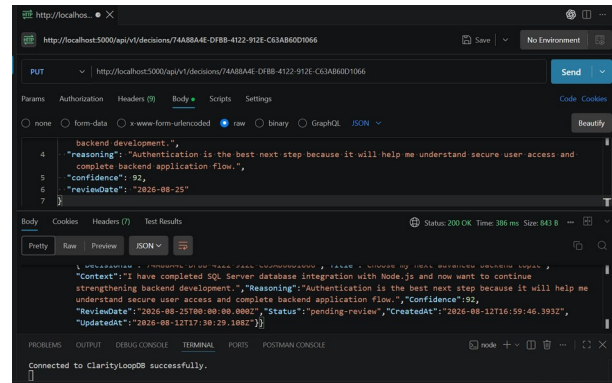


Figure 11. PUT operation updates a decision.

9. Evidence Screenshots (continued)

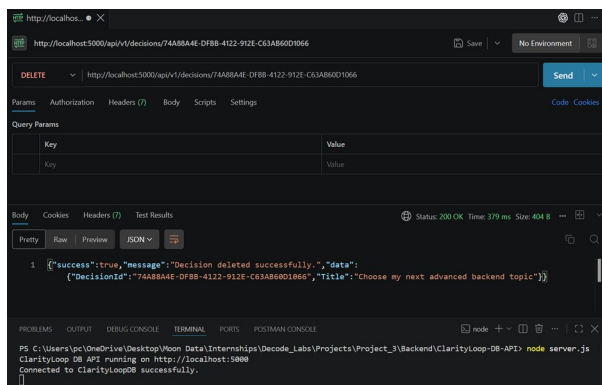


Figure 12. DELETE operation removes a decision.

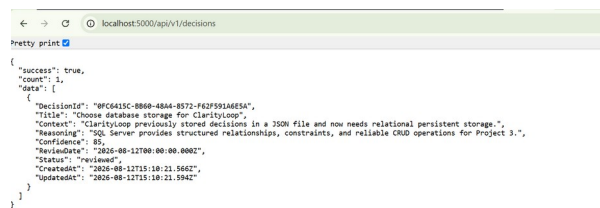


Figure 13. Verification after deletion.

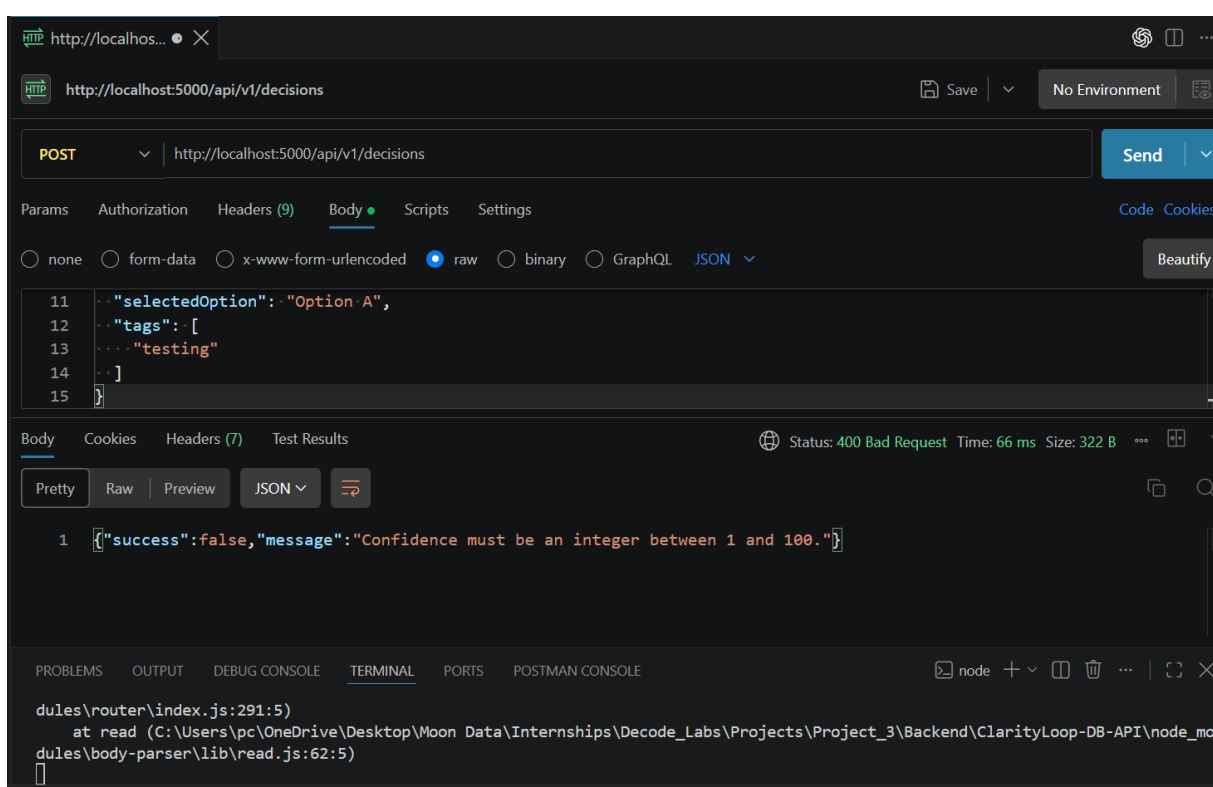


Figure 14. Controlled 400 Bad Request response for invalid input.

10. Key Engineering Decisions and Challenges

10.1 Moving from Flat Files to Relationships

A decision is not an isolated record: it can have several options, one later review and several reusable tags. The relational model separates these concerns into focused tables so repeated data is minimized and each relationship has an explicit integrity rule.

10.2 Preserving Multi-table Consistency

CREATE spans the Decisions, DecisionOptions, Tags and DecisionTags tables. A transaction ensures that partial records are not left behind if any related insert fails.

10.3 Safe Input Handling

The backend validates required fields and domain ranges before reaching SQL Server, then passes application data through typed SQL inputs. Database constraints provide a second integrity layer.

10.4 Deletion Consistency

Cascading foreign keys allow a decision to be deleted without leaving orphaned option, review or tag-association rows.

10.5 Local Connectivity

The implementation uses a local SQL Server instance with Windows Authentication, msnodesqlv8 and ODBC Driver 18. This is appropriate for local training and testing but must be reconfigured for cloud deployment.

11. Learning Outcomes

- Designed and implemented a normalized relational schema for a multi-entity backend domain.
- Applied primary keys, foreign keys, check constraints, unique constraints and cascading behavior.
- Connected an Express application to Microsoft SQL Server using a reusable connection pool.
- Implemented full CRUD operations with appropriate HTTP status codes.
- Used transactions to protect multi-step writes.
- Used typed SQL parameters and application-level validation for safer data handling.
- Tested database behavior and API endpoints using SSMS and Postman.
- Prepared SQL scripts, API documentation, a Postman collection and a GitHub-ready repository.

11.2 Future Enhancements

- Add authentication and authorization.
- Add GET /api/v1/decisions/:id.
- Add dedicated review endpoints and richer relationship retrieval.
- Add pagination, sorting and filtering for decision lists.
- Create automated unit and integration tests.
- Move database configuration to deployment-safe environment variables.
- Deploy SQL Server and the API to a cloud environment.
- Build a frontend dashboard that consumes the REST API.

12. Conclusion

ClarityLoop DB successfully completes the Decode Labs Project 3 database-integration milestone. The project connects a Node.js/Express backend to Microsoft SQL Server, replaces temporary file-based persistence with a relational schema, and implements persistent CRUD operations.

The implementation demonstrates more than simple connectivity: relationships are modeled explicitly, database rules enforce valid data, multi-table writes are transactional, application values are parameterized, and invalid requests receive controlled error responses. Manual evidence confirms successful schema creation, relationship queries, database connectivity, CRUD operations, deletion behavior and validation.

Final Assessment

The completed system provides a reliable foundation for the next backend milestones, including authentication, advanced query features, automated testing, cloud deployment and frontend integration.

References and Project Artifacts

12. Decode Labs - Project 3 Industrial Training Kit, Full Stack Development, Batch 2026.
13. ClarityLoopDB_Schema.sql - database tables, relationships and constraints.
14. ClarityLoopDB_TestData.sql and ClarityLoopDB_Verification.sql - test and verification scripts.
15. server.js and src/db.js - Express API and SQL Server connectivity implementation.
16. ClarityLoop_DB_API_Project3.postman_collection.json - API test collection.
17. Project 3 screenshot set P3_SS_01 through P3_SS_11 - implementation evidence.

Appendix A - Setup and Execution Guide

A.1 Prerequisites

- Node.js and npm.
- Microsoft SQL Server.
- SQL Server Management Studio.
- ODBC Driver 18 for SQL Server.
- Postman or another REST client.

A.2 Database Setup

18. Open SQL Server Management Studio.
19. Run database/ClarityLoopDB_Schema.sql.
20. Optionally run database/ClarityLoopDB_TestData.sql.
21. Run database/ClarityLoopDB_Verification.sql to confirm records and relationships.

A.3 Backend Setup

```
npm install
node test-db.js
node server.js
```

The local API listens on `http://localhost:5000`.

A.4 Example CREATE Request

```
POST /api/v1/decisions
Content-Type: application/json

{
  "title": "Choose my next backend learning topic",
  "context": "I have connected Node.js with SQL Server.",
  "reasoning": "Authentication is the next logical backend milestone.",
  "confidence": 88,
  "reviewDate": "2026-08-20",
  "options": [
    "Learn authentication",
    "Learn caching",
    "Learn WebSockets"
  ],
  "selectedOption": "Learn authentication",
  "tags": ["backend", "learning"]
}
```

Appendix B - Submission Repository Structure

```
Project-3-ClarityLoop-DB/
|
+-- database/
|   +-- ClarityLoopDB_Schema.sql
|   +-- ClarityLoopDB_TestData.sql
|   +-- ClarityLoopDB_Verification.sql
|
+-- docs/
|   +-- API_DOCUMENTATION.md
|   +-- DATABASE_DESIGN.md
|   +-- PROJECT_SUMMARY.md
|
+-- postman/
|   +-- ClarityLoop_DB_API_Project3.postman_collection.json
|
+-- screenshots/
|   +-- P3_SS_01_Database_Tables.png
|   +-- ... P3_SS_11_Validation_400.png
|
+-- src/
|   +-- db.js
|
+-- .env.example
+-- .gitignore
+-- package.json
+-- package-lock.json
+-- README.md
+-- server.js
+-- test-db.js
```

Submission hygiene

node_modules and the real .env file must not be uploaded to GitHub. The repository should contain source code, scripts, documentation, screenshots and safe configuration examples only.